

Distributed Configuration Store (DCS) Tools: A Deep Technical Survey and Ranking for PostgreSQL HA Practitioners

TL;DR

- **etcd is the single most widely deployed DCS and the default, best-supported backend for PostgreSQL HA via Patroni** — it is a Raft-based, strongly consistent, gRPC key-value store (~51.9k GitHub stars, CNCF-graduated, and the storage engine of every Kubernetes cluster). For most Postgres HA clusters, use etcd (or the Kubernetes API when running on K8s).
- Among open-source DCS options, the "Patroni-capable" tier is **etcd, Consul, ZooKeeper, Kubernetes API, and Patroni's built-in Raft (pysyncobj, now deprecated)**; Consul's 2023 relicensing to BUSL (unlike Terraform→OpenTofu and Vault→OpenBao, Consul got no prominent community fork) is the main strategic caveat for that option.
- Tools like Nacos, Eureka, Redis/Valkey, Spring Cloud Config, TiKV, FoundationDB, Serf, Doozerd, and Chubby occupy adjacent niches (service discovery, app config, coordination, storage layers) but are **not** general Patroni DCS backends; choose them based on ecosystem fit, not Postgres HA.

Key Findings

- **Consensus is the dividing line.** A usable DCS for leader election must be strongly consistent (linearizable, CP under CAP). etcd, ZooKeeper, and Consul all qualify. Redis, Eureka, and default Nacos are AP/eventually consistent and are unsafe as the sole arbiter of "who is primary."
 - **Patroni officially supports five DCS backends:** etcd (v2/v3), Consul, ZooKeeper (and Exhibitor), the Kubernetes API, and a self-contained Raft implementation via the `(pysyncobj)` Python module. The Raft/pysyncobj option is now deprecated — Patroni's own release notes state: *"The RAFT support is deprecated. We will do our best to maintain it, but take neither guarantee nor responsibility for possible issues."*
 - **etcd dominates by adoption.** It is CNCF-graduated (accepted as incubating December 11, 2018; graduated November 24, 2020), backs Kubernetes, and has by far the largest star count and production footprint of any purpose-built DCS.
 - **Licensing has reshaped the landscape.** HashiCorp moved Consul to BUSL 1.1 on August 10, 2023; Redis moved from BSD to SSPL/RSALv2 effective Redis 7.4 (announced March 20, 2024), adding AGPLv3 as a third option in Redis 8.0 GA (May 1, 2025), which triggered the Linux Foundation's Valkey fork. etcd, ZooKeeper, TiKV, FoundationDB, Nacos, Eureka, and Spring Cloud Config remain permissively licensed (Apache 2.0).
 - **Language split is clean:** Go (etcd, Consul, Serf, Doozerd), Java (ZooKeeper, Nacos, Eureka, Curator, Spring Cloud Config), Rust (TiKV), C/C++/Flow (Redis/Valkey, FoundationDB), Python (Patroni itself).
-

Details

Part A — Open-Source DCS Tools

1. etcd — the reference DCS (Rank #1 by adoption)

What it is / how it works. etcd is a distributed, strongly consistent key-value store built on the **Raft** consensus algorithm. A cluster elects a single leader that serializes all writes; a write is committed once a majority (quorum) of members persist it to their write-ahead log on disk, after which it is applied to the replicated state machine. Reads are linearizable (via ReadIndex) or can be served as cheaper serializable reads. It uses an MVCC model over a flat, ordered keyspace with a global monotonically increasing revision number, persisted through a WAL plus periodic snapshots into a bbolt (B+tree) backend. Clients use a **gRPC** API (etcd v3) and can `(watch)` key ranges to receive streamed change notifications, plus leases/TTLs for ephemeral keys — exactly the primitives Patroni needs for the leader key. `(CNCF)` `(TiKV)`

- **Consistency:** linearizable, CP. Requires quorum; a partitioned minority cannot serve writes or linearizable reads.
- **Use cases:** Kubernetes' primary datastore; service discovery; distributed locks/leader election; configuration storage; **the default and most battle-tested Patroni DCS backend.**
- **Based on:** Raft; bbolt/WAL storage; gRPC + HTTP (JSON gateway).
- **Language:** Go.
- **Repo:** <https://github.com/etcd-io/etcd> (~51.9k stars).
- **Pros:** Strong consistency; excellent watch/lease semantics; huge community; CNCF-graduated (graduated November 24, 2020); Apache 2.0; RBAC + TLS/mTLS. Simplest choice when already running Kubernetes.
- **Cons:** Quorum latency is sensitive to disk/network fsync latency; write throughput bounded by leader; keyspace not ideal for very large datasets; needs an odd node count (3/5) and operational care (compaction, defrag, snapshots).

2. Apache ZooKeeper — the veteran coordinator (Rank #2-3)

What it is / how it works. ZooKeeper is a distributed coordination service exposing a hierarchical filesystem-like namespace of **znodes**. It uses the **ZAB (ZooKeeper Atomic Broadcast)** protocol — a leader-based primary-backup atomic broadcast (a two-phase-commit-like protocol, comparable in role to Paxos/Raft) — to guarantee linearizable writes and FIFO client order. Writes go to the leader and are broadcast to followers; reads are served locally by any server (so reads can be stale unless a `sync` is issued). Data is held in memory (fast reads) with transaction logs + snapshots on disk. **Ephemeral znodes** (auto-deleted when a client session/heartbeat ends) and **sequential znodes** + **watches** are the classic building blocks for leader election, locks, and fencing tokens. Medium

- **Consistency:** linearizable writes, sequential consistency; CP.
- **Use cases:** coordination for Hadoop, Kafka (pre-KRaft), HBase, Solr; leader election; distributed locks; config; a supported **Patroni DCS backend** (via kazoo; Exhibitor also supported). Medium Apache
- **Based on:** ZAB; in-memory DB + WAL/snapshots; custom binary protocol (Jute).
- **Language:** Java.
- **Repo:** <https://github.com/apache/zookeeper> (~12.8k stars).
- **Pros:** Extremely mature and proven; strong consistency; rich recipes (via Curator); good where an org already runs the JVM/Hadoop/Kafka stack.
- **Cons:** Operationally heavier (JVM tuning, GC, memory sizing); clients must maintain persistent sessions/heartbeats; hierarchical API is lower-level; watches are one-shot; generally considered more complex to operate than etcd for a greenfield Postgres cluster.

3. HashiCorp Consul — service mesh + DCS, now BUSL (Rank #2-3)

What it is / how it works. Consul is a service-networking platform combining service discovery, health checking, a KV store, and a full service mesh (Consul Connect/Envoy). Its server agents form a **Raft** quorum for strongly consistent state (the KV store and catalog), while **Serf** (an implementation of the **SWIM** gossip protocol) handles cluster membership, failure detection, and event propagation across both LAN and WAN pools. Client agents are lightweight and forward RPCs to servers. It exposes HTTP and DNS APIs; blocking queries provide watch-like long-polling.

- **Consistency:** Raft-backed KV is linearizable/CP; gossip membership is eventually consistent (AP) by design.
- **Use cases:** service discovery/mesh, health checks, multi-datacenter, KV config; a supported **Patroni DCS backend** (with automatic `register_service` of primary/replica), attractive when Consul is already operated for service mesh. GitHub
- **Based on:** Raft (consistency) + Serf/SWIM gossip (membership); Go.
- **Language:** Go.
- **Repo:** <https://github.com/hashicorp/consul> (~30k stars).
- **Pros:** Native multi-DC federation; built-in health checks and DNS; service mesh; good for consolidating tooling when Consul already exists. GitHub
- **Cons — critical licensing note:** On August 10, 2023, HashiCorp (NASDAQ: HCP) changed its source code license from Mozilla Public License v2.0 to the Business Source License v1.1 (BSL/BUSL) across Terraform, Vault, Consul, Boundary, Nomad, Waypoint, Packer, and Vagrant. BUSL is not OSI-approved and forbids using the software to build a competitive offering. **The last MPL-licensed Consul release was 1.16.1**, with security fixes backported under MPL 2.0 only through December 31, 2023. Crucially — and unlike Terraform (forked to **OpenTofu**, accepted into the Linux Foundation on September 20, 2023 after its manifesto drew 32K+ GitHub stars) and Vault (forked to **OpenBao**, an MPL, Linux-Foundation, IBM-initiated Vault fork) — **Consul has received no prominent community fork**. OpenBao even dropped Consul as a storage backend option because Consul is no longer OSI-licensed. Operationally, Consul is also heavier than etcd for a pure Postgres DCS, and gossip is sensitive to UDP packet loss.

4. Kubernetes API (as a Patroni DCS)

What it is / how it works. When running Patroni on Kubernetes, the Kubernetes API server itself (backed by etcd) can serve as the DCS: Patroni stores leader/member state in Kubernetes objects (Endpoints/ConfigMaps or a Custom Resource) and uses Kubernetes' optimistic-concurrency (resourceVersion CAS) and leases for leader election. This avoids running a *second* etcd cluster alongside the one already backing K8s.

- **Consistency:** inherits etcd's linearizable/CP guarantees via the API server.
- **Use cases:** Postgres HA operators (Zalando, Crunchy, CloudNativePG-adjacent patterns) on Kubernetes.
- **Based on:** Kubernetes API → etcd/Raft; Go (Patroni integration in Python).
- **Repo:** <https://github.com/kubernetes/kubernetes> (API); Patroni: <https://github.com/patroni/patroni>.
- **Pros:** No extra moving parts on K8s; native RBAC; the standard choice for containerized Postgres HA.
- **Cons:** Only meaningful inside Kubernetes; adds API-server load; watch/lease semantics mediated by K8s.

5. Patroni built-in Raft (pysyncobj) — DCS-less option

What it is / how it works. Patroni 2.0 (September 2020) introduced the ability to run **without any external DCS** by embedding a pure-Python Raft implementation (`pysyncobj`). Patroni nodes form their own Raft consensus; a third lightweight (`patroni-raft-controller`) can act as a voting member so two DB nodes plus one controller achieve quorum.

- **Based on:** Raft via `pysyncobj`; Python.
- **Repo:** <https://github.com/patroni/patroni>.
- **Pros:** No third-party dependency; simplest to stand up for small/edge clusters; supports 2-node + controller topologies.
- **Cons:** **Officially deprecated** — Patroni's release notes state support is "best effort" with "neither guarantee nor responsibility for possible issues"; less battle-tested than etcd; not recommended for large or critical production fleets.

6. TiKV — distributed transactional KV (adjacent, Rust)

What it is / how it works. TiKV is a distributed transactional key-value store (the storage layer of TiDB), CNCF-**graduated**. It shards data into **Regions**, each a **Raft** group replicated across nodes, with **RocksDB** as the local storage engine and a **Placement Driver (PD)** for scheduling/balancing. It implements Google Percolator-style distributed transactions (2PC over Raft) and can offer Spanner-like externally consistent transactions. Communication is via **gRPC**. [GitHub + 3](#)

- **Consistency:** strongly consistent per-Region via Raft; supports snapshot isolation / externally consistent txns.
- **Use cases:** massive-scale transactional KV storage, TiDB backend, horizontally scalable metadata stores — **not** a Patroni DCS.
- **Based on:** Raft (raft-rs, derived from etcd's algorithm) + RocksDB + gRPC; Rust.
- **Repo:** <https://github.com/tikv/tikv> (~16.7k stars).
- **Pros:** Scales to 100+ TB; strong consistency; ACID transactions; Apache 2.0; CNCF-graduated. [GitHub](#)
- **Cons:** Heavyweight; overkill for coordination; not designed as a lightweight leader-election store.

7. FoundationDB — ordered transactional KV (adjacent, Flow/C++)

What it is / how it works. FoundationDB (FDB), open-sourced by Apple in April 2018 (Apache 2.0), is an ordered, strictly serializable, ACID key-value store with an **unbundled architecture**: a control plane whose **Coordinators form an Active Disk Paxos group** to elect a ClusterController and persist metadata, plus a data plane of stateless transaction processes (Sequencer, Proxies, Resolvers) and stateful LogServers/StorageServers. Notably, FDB does **not** use Paxos/Raft to replicate its transaction logs — it synchronously writes to all replicas and eagerly detects/recovers from failures, tolerating f failures with only $f+1$ replicas. It combines OCC + MVCC for strict serializability and is famous for its deterministic simulation testing. (FDB's early versions actually used ZooKeeper for coordination, later replaced by a bespoke Flow Paxos implementation.) Data models (SQL, document, etc.) are built as "layers" over the core. [Wikipedia](#) [GitHub](#)

- **Consistency:** strictly serializable; Paxos only for control-plane metadata/coordination.
- **Use cases:** foundational storage layer at Apple, Snowflake, etc.; multi-model backends — **not** a Patroni DCS. [Blogger](#)
- **Based on:** Active Disk Paxos (control plane) + OCC/MVCC transactions; written in **Flow** (a C++ extension for async/actor programming). [GitHub](#)
- **Repo:** <https://github.com/apple/foundationdb> (~16.5k stars).
- **Pros:** Exceptional reliability/testing rigor; strict serializability; efficient failure model.
- **Cons:** Steep learning curve; layered model pushes complexity to clients; not a coordination primitive.

8. Redis / Valkey — coordination by convention (adjacent, C)

What it is / how it works. Redis is an in-memory data-structure store often used for locks (`SET NX PX`, Redlock), leader hints, and ephemeral coordination. It is fast but its default replication is **asynchronous** and it is **not** a linearizable consensus system; Redlock in particular is contested for correctness under partitions. Redis Sentinel and Redis Cluster add failover/sharding but do not make Redis a safe strong-consistency arbiter for a database primary.

- **Consistency:** effectively AP / eventually consistent for coordination purposes; unsafe as a sole DCS for Postgres primary election.
- **Use cases:** caching, queues, rate limiting, best-effort locks/coordination — **not** a Patroni DCS backend.
- **Based on:** single-threaded command core (multi-threaded I/O in Valkey 8+); C.
- **Repos:** Redis <https://github.com/redis/redis> (~75k stars); **Valkey** <https://github.com/valkey-io/valkey> (~26k stars).
- **Licensing (major 2024 event):** Redis left BSD-3-Clause for a dual **SSPL/RSALv2** model effective Redis 7.4 (announced March 20, 2024); neither license is OSI-approved. Per redis.io, "Redis 7.2.x...remain subject to the BSD3 license. Redis Community Edition 7.4.x to 7.8.x remain subject to the dual RSALv2/SSPLv1 license." The Linux Foundation announced **Valkey** on March 28, 2024, forking the last BSD release (7.2.4) with support from **AWS, Google Cloud, Oracle, Ericsson, and Snap Inc.** **Redis 8.0 GA (May 1, 2025)** re-added the OSI-approved **AGPLv3** as a third license option, making Redis open-source again under that option; Valkey remains BSD-3.

- **Pros:** Extremely fast; ubiquitous; Valkey restores permissive licensing and adds multi-threaded I/O.
- **Cons:** Not strongly consistent — do not use as the authoritative leader store for Postgres HA.

9. Nacos — service discovery + config (adjacent, Java)

What it is / how it works. Nacos (Alibaba) unifies dynamic service discovery, configuration management, and service metadata. It uses a hybrid consistency model: **Raft (JRaft)** for strongly consistent (CP) data and the **Distro** protocol for AP, eventually consistent service-registration data; it can be deployed CP or AP. Config data is typically persisted to a relational DB (e.g., MySQL). APIs are HTTP + SDKs (with gRPC push in newer versions). [\(Medium\)](#)

- **Use cases:** microservice registry + config center, especially in the Spring Cloud Alibaba / Dubbo ecosystem — **not** a standard Patroni DCS.
- **Based on:** Raft (JRaft) + Distro; MySQL persistence; HTTP/gRPC; Java.
- **Repo:** <https://github.com/alibaba/nacos> (~32.7k stars).
- **Pros:** All-in-one discovery+config; dynamic push; strong in the Alibaba/Java ecosystem; Apache 2.0.
- **Cons:** DB dependency adds operational weight; not designed for Postgres leader election.

10. Netflix Eureka — AP service registry (adjacent, Java)

What it is / how it works. Eureka is a REST-based service registry from Netflix OSS, designed for **AP** (availability over consistency) with client-side caching and a "self-preservation" mode that tolerates network partitions rather than expiring registrations. There is no consensus algorithm; peers replicate registrations best-effort.

- **Use cases:** service discovery in Spring Cloud Netflix / AWS microservices — **not** a DCS for leader election.
- **Based on:** peer replication, client-side load balancing; no consensus; Java.
- **Repo:** <https://github.com/Netflix/eureka> (~12.7k stars).
- **Maintenance:** The original **Eureka 2.x re-architecture was discontinued/abandoned**; **Eureka 1.x remains the actively maintained core**. A separate current "2.0.x" line (latest ~v2.0.6, 2026) exists purely for Spring Boot 3 / Jakarta EE 9 (javax→jakarta) compatibility and is unrelated to the abandoned rewrite. [\(GitHub\)](#) [\(GitHub\)](#)
- **Pros:** Simple, resilient, plug-and-play in Spring Cloud Netflix.
- **Cons:** Eventually consistent — structurally unsuitable as a strong-consistency DCS.

Honorable mentions (open source)

- **Apache Curator** (<https://github.com/apache/curator>, ~3.2k stars; Java, Apache 2.0): not a DCS itself but the *de facto* high-level **client framework for ZooKeeper**, providing tested recipes (leader election, locks, caches). Relevant because it makes ZooKeeper practical.
- **HashiCorp Serf** (<https://github.com/hashicorp/serf>, ~6.1k stars; Go): the **SWIM gossip** membership/failure-detection library underpinning Consul. It is AP (eventually consistent) and provides *no* strong consistency or KV store — not a DCS on its own. Now also under BUSL.
- **Doozerd** (Go): an early Paxos-based highly-available data store that was an influence/contemporary of etcd; effectively unmaintained today and of historical interest only.

Part B — Commercial / Proprietary DCS Tools

Google Chubby — the proprietary ancestor

What it is / how it works. Chubby is Google's internal **coarse-grained lock service** and small-file store for loosely-coupled distributed systems. A Chubby "cell" is typically **5 replicas that run Paxos** to elect a master; the master handles all reads/writes. It exposes a filesystem-like namespace of small files with advisory locks, ephemeral nodes, event notifications, and consistent client caching. It targets low-rate, long-held (hours/days) locks — its dominant real use turned out to be **name service, config storage, and master election** (e.g., for GFS, Bigtable). Chubby directly inspired ZooKeeper and, indirectly, etcd.

- **Consistency:** strong (Paxos); CP.
- **Based on:** Paxos; C++ (internal).
- **Availability:** **proprietary, Google-internal** — **not open source**, no public repo.
- **Relevance:** the conceptual template for every DCS in Part A.

Cloud-managed configuration stores (proprietary services)

These are managed configuration/parameter services. They are excellent for **application configuration and secrets** but are **not** designed as strongly-consistent leader-election stores and are **not** supported Patroni DCS backends. (Patroni does bundle AWS *callbacks* via boto3, but

that is for instance-tagging/EIP failover actions, not for using Parameter Store as the consensus store.)

- **AWS Systems Manager Parameter Store** — hierarchical key/value store (String, StringList, SecureString via KMS), IAM-controlled, versioned (retains the 100 most recent versions), integrates with EC2/Lambda/CloudFormation. Pull-based (values can be stale between reads). Standard tier free; advanced tier priced. Proprietary AWS service. (Be a Better Dev)
- **AWS AppConfig** (part of Systems Manager) — an "enhanced" config service adding **validated, staged/canary/linear rollouts, automatic rollback, and feature flags**; can source data from its hosted store, Parameter Store, Secrets Manager, or S3. Proprietary.
- **AWS Secrets Manager** — purpose-built for secrets with automatic rotation, cross-region replication, and KMS encryption (adjacent to config but secret-focused). Proprietary.
- **Azure App Configuration** — Microsoft's managed centralized config + feature-flag service, with point-in-time snapshots and Key Vault integration for secrets. Proprietary.
- **Google Cloud Runtime Configurator (RuntimeConfig)** — GCP's managed dynamic key/value + watcher/waiter service for runtime config and startup coordination. Proprietary, and notably a lower-profile/legacy offering.
- **Spring Cloud Config Server** — technically **open source** (Apache 2.0, <https://github.com/spring-cloud/spring-cloud-config>, ~2.0k stars) but usually grouped with these because it plays the same role: a Spring client/server config service with a **Git backend** (also SVN, filesystem "native," Vault, JDBC), serving versioned config over HTTP with symmetric/asymmetric encryption of values. It is a config-distribution tool, **not** a consensus/leader-election DCS.

Master Comparison Table

Tool	Consensus / Model	Language	License	Repository	Typical use cases	Patroni DCS backend?
etcd	Raft (linearizable, CP)	Go	Apache 2.0	github.com/etcd-io/etcd	K8s store, leader election, config, service discovery	Yes — default/recommended
ZooKeeper	ZAB (linearizable writes, CP)	Java	Apache 2.0	github.com/apache/zookeeper	Coordination (Kafka/Hadoop), locks, election	Yes (kazoo / Exhibitor)
Consul	Raft (KV, CP) + Serf/SWIM gossip (AP)	Go	BUSL 1.1 (was MPL 2.0 ≤1.16.1)	github.com/hashicorp/consul	Service mesh/discovery, health checks, multi-DC, KV	Yes
Kubernetes API	via etcd/Raft (CP)	Go (Patroni: Python)	Apache 2.0	github.com/kubernetes/kubernetes	Postgres HA operators on K8s	Yes (on Kubernetes)
Patroni Raft (pysyncobj)	Raft (CP)	Python	MIT	github.com/patroni/patroni	DCS-less small/edge Postgres HA	Yes — but deprecated
TiKV	Raft per-Region (CP) + Percolator txns	Rust	Apache 2.0	github.com/tikv/tikv	Distributed transactional storage (TiDB)	No
FoundationDB	Active Disk Paxos (control) + OCC/MVCC	Flow/C++	Apache 2.0	github.com/apple/foundationdb	Strictly serializable storage layer	No
Redis / Valkey	Async replication (AP; not consensus)	C	Redis: SSPL/RSA Lv2/AGPL v3 (8.0+); Valkey: BSD-3	github.com/redis/redis · github.com/valkey-io/valkey	Cache, queues, best-effort locks	No (unsafe)

Tool	Consensus / Model	Language	License	Repository	Typical use cases	Patroni DCS backend?
Nacos	Raft (JRaft, CP) + Distro (AP)	Java	Apache 2.0	github.com/alibaba/nacos	Service registry + config center	No
Eureka	Best-effort peer replication (AP)	Java	Apache 2.0	github.com/Netflix/eureka	Service discovery (Spring Cloud Netflix)	No
Curator	(ZooKeeper client framework)	Java	Apache 2.0	github.com/apache/curator	ZooKeeper recipes (election/locks)	No (client lib)
Serf	SWIM gossip (AP)	Go	BUSL 1.1	github.com/hashicorp/serf	Membership/failure detection (under Consul)	No
Chubby	Paxos (CP)	C++	Proprietary (Google-internal)	—	Locks, naming, master election (GFS/Bigtable)	No
AWS Parameter Store	Managed (not consensus)	—	Proprietary	—	App config / lightweight secrets	No
AWS AppConfig	Managed (staged rollouts)	—	Proprietary	—	Feature flags, dynamic config	No
Azure App Configuration	Managed	—	Proprietary	—	Centralized config + feature flags	No
GCP RuntimeConfig	Managed	—	Proprietary	—	Runtime config, startup coordination	No
Spring Cloud Config	Git-backed (not consensus)	Java	Apache 2.0	github.com/spring-cloud/spring-cloud-config	Spring app config distribution	No

Adoption Ranking (by evidence)

Ranking blends purpose-built-DCS status, production footprint, CNCF status, and GitHub stars (2026 snapshots; star counts are UI-rounded and fluctuate):

Rank	Tool	Stars (~2026)	Why
1	etcd	~51.9k	Backs all of Kubernetes; CNCF-graduated; default Patroni DCS.
2	Consul	~30k	Ubiquitous service mesh/DCS; Patroni backend; tempered by BUSL.
3	ZooKeeper	~12.8k	Decades of production coordination (Kafka/Hadoop); Patroni backend.
4	Nacos	~32.7k	Huge in the China/Java microservice ecosystem (stars > ZK, but narrower as a DCS).
5	TiKV	~16.7k	CNCF-graduated storage layer; adjacent to DCS.
6	FoundationDB	~16.5k	Elite reliability; storage layer, not coordination.
7	Eureka	~12.7k	Widely used AP registry; not a strong-consistency DCS.
8	Curator	~3.2k	Essential ZooKeeper client framework.
9	Serf	~6.1k	Gossip membership library under Consul.
10	Spring Cloud Config	~2.0k	Popular app-config server (not a consensus DCS).

(Redis at ~75k and Valkey at ~26k dwarf all of these by stars, but as coordination tools they are non-consensus and thus ranked separately/excluded from the "DCS proper" ordering.)

Recommendations

For a new PostgreSQL HA cluster (Patroni), staged guidance:

1. **Default / start here: etcd.** It is the reference DCS, the best-tested Patroni backend, strongly consistent, and well documented. Run a dedicated **3-node** etcd cluster (5 nodes for higher failure tolerance / multi-rack), on SSDs, physically separate from the Postgres nodes. Use TLS/mTLS and RBAC. This is the right default for the large majority of on-prem and VM deployments.
2. **If you run on Kubernetes: use the Kubernetes API backend.** Don't stand up a second etcd — let Patroni store state in the K8s API (backed by the cluster's etcd). This is the standard for Zalando/Crunchy-style Postgres operators.
3. **If you already operate Consul** for service mesh/discovery: using Consul as the Patroni DCS is legitimate and consolidates tooling — **but** factor in the BUSL license (no community fork exists) and enable ACLs + TLS. Don't adopt Consul *solely* to be a Postgres DCS; etcd is lighter for that.
4. **If you are embedded in a JVM/Kafka/Hadoop shop** already running ZooKeeper: ZooKeeper is a fine Patroni backend (use kazoo/Exhibitor). Otherwise avoid the extra JVM operational burden for greenfield Postgres.
5. **Small/edge/2-node clusters with no appetite for a separate DCS:** Patroni's built-in Raft (pysyncobj) with a `patroni_raft_controller` works, but note it is **deprecated** — treat it as a stopgap, not a strategic choice, and plan to migrate to etcd if the fleet grows.

Do NOT use as your Postgres leader-election DCS: Redis/Valkey, Eureka, default-AP Nacos, or cloud config stores (Parameter Store/AppConfig/App Configuration/RuntimeConfig/Spring Cloud Config). These lack the linearizable consensus guarantees required to prevent split-brain. They are appropriate for their own domains (caching, service discovery, app configuration, feature flags).

Thresholds that change the recommendation:

- **Moving to Kubernetes** → switch from standalone etcd to the K8s API backend.
- **etcd write latency / fsync pressure rising** (large fleets, many Patroni clusters) → scale to 5 etcd members, move to faster disks, and shard Patroni clusters across DCS namespaces rather than adding etcd nodes indefinitely.
- **Consul BUSL becomes a procurement/legal blocker** → migrate the DCS to etcd (Patroni makes this a config change, not a re-architecture).
- **Multi-datacenter DCS requirement** → Consul's native WAN federation is a genuine advantage; otherwise keep the DCS within one low-latency failure domain and use Postgres-level standby clusters across DCs.

Caveats

- **GitHub star counts are UI-rounded 2026 snapshots and fluctuate daily;** they indicate rough popularity, not production adoption, and should not be over-interpreted (e.g., Nacos > ZooKeeper by stars but ZooKeeper has far deeper coordination-in-production history).

- **CAP framing is a simplification.** etcd/ZooKeeper/Consul-KV are CP; Redis/Eureka/Nacos-AP trade consistency for availability. The safety argument for Postgres HA rests on linearizability of the leader key, which only the CP systems provide.
- **FoundationDB's Paxos** applies only to control-plane coordination; its data-plane replication deliberately avoids quorum consensus — a nuance often misreported.
- **Redis licensing is a moving target:** BSD ($\leq 7.2.4$) $\rightarrow$ SSPL/RSALv2 (7.4+) $\rightarrow$ tri-license adding AGPLv3 (8.0 GA, May 1, 2025). Valkey remains BSD-3. Confirm the exact version's license before deploying.
- **Consul "no fork" is an absence-of-evidence finding:** as of research, no OpenTofu/OpenBao-equivalent community fork of Consul had achieved prominence; a minor fork could exist unnoticed. OpenBao is a fork of **Vault**, not Consul.
- **Chubby specifics** (Paxos, 5-replica cells, coarse-grained locks) come from Google's published 2006 paper and secondary summaries; it remains closed-source, so current internal details are not public.
- Doozerd is included for historical completeness; it is effectively unmaintained and should not be selected for new work.